

Agentes LLM

Ulises Moya

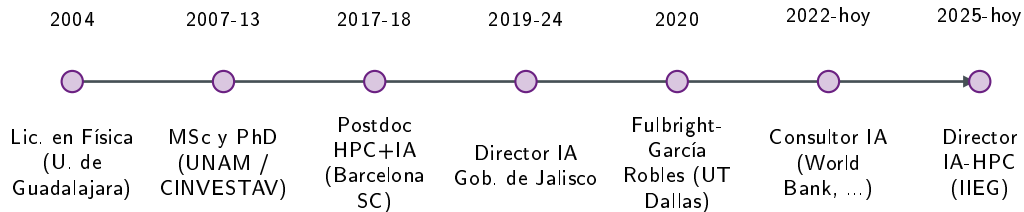
IIEG, TSJ-Mario Molina

3 de julio de 2026

Contenido

1. ¿Qué es un agente?
2. Antecedentes e historia
3. Aprendizaje por refuerzo
4. Agente clásico vs. agente LLM
5. Tipos de modelos LLM
6. RAG: Retrieval-Augmented Generation
7. Chain-of-Thought y ReAct
8. Automatizar procesos de código con agentes
9. Frameworks de agentes
10. Sistemas multiagente
11. Patrones de diseño agénticos
12. Herramientas de agentes
13. Demo: un agente simple con LLM local

Sobre mí: trayectoria



Ph.D. en Ingeniería Eléctrica · +65 publicaciones en IA para salud, clima y NLP
Top 10 UNESCO-IRCAI Global 100 · Proyectos con OECD-GPAI, Banco Mundial y ONU

Equipo de Trabajo



¿Qué tienen en común un termostato,
un jugador de ajedrez artificial
y ChatGPT usando un navegador?

¿Qué tienen en común un termostato,
un jugador de ajedrez artificial
y ChatGPT usando un navegador?

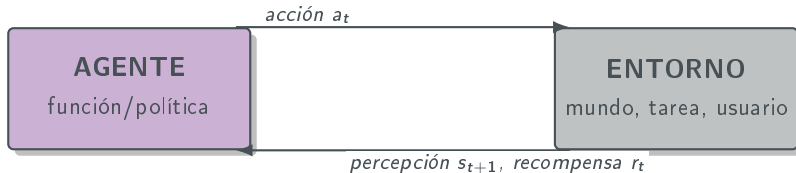
Los tres pueden describirse como **agentes**.

Agente (Russell & Norvig)

Un agente es cualquier entidad que **percibe** su entorno a través de sensores y **actúa** sobre ese entorno a través de actuadores, con el fin de alcanzar un objetivo.

- No requiere “inteligencia” para ser agente: un termostato ya califica.
- Lo que lo hace *interesante* es qué tan racional es su comportamiento frente al objetivo.
- Un **agente racional** elige, en cada instante, la acción que maximiza su desempeño esperado dado lo que ha percibido.

El ciclo agente-entorno



- Sensores → estado percibido.
- Programa/política del agente → decide la acción.
- Actuadores → ejecutan la acción sobre el entorno.
- El entorno cambia, y el ciclo se repite.

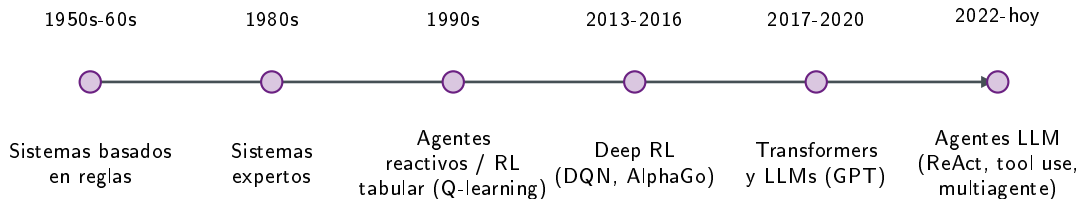
Tipos clásicos de agentes (según su arquitectura)

- **Reactivo simple**: regla condición-acción, sin memoria (termostato).
- **Basado en modelo**: mantiene un estado interno del mundo.
- **Basado en objetivos**: elige acciones que acercan a una meta.
- **Basado en utilidad**: pondera múltiples objetivos en conflicto.
- **Que aprende**: mejora su política con la experiencia (RL).

Adelanto

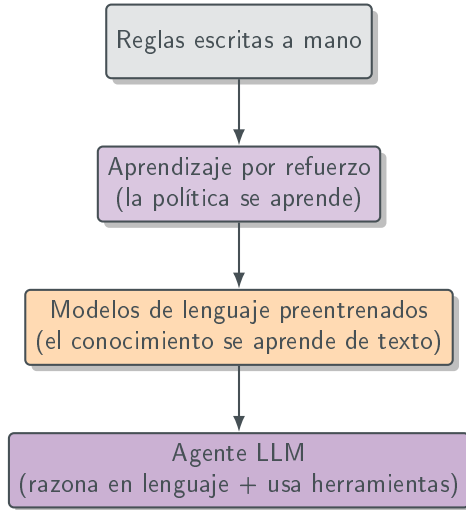
Un **agente LLM** añade una capa nueva: usa lenguaje natural como espacio de razonamiento interno, no solo como entrada/salida.

Línea de tiempo: de reglas a agentes que razonan



- **Sistemas expertos** (MYCIN, DENDRAL): reglas si-entonces escritas a mano por expertos humanos.
- **Planificación clásica** (STRIPS): buscar una secuencia de acciones que lleve de un estado inicial a una meta.
- **Agentes reactivos** (Brooks, robótica): sin representación interna del mundo, solo estímulo-respuesta.
- Limitación común: **frágiles** fuera del dominio para el que fueron programados; no generalizan.

El salto: aprendizaje y luego lenguaje



Hitos recientes que habilitaron los agentes LLM

- **2017** — *Attention is All You Need*: arquitectura Transformer.
- **2020-2022** — GPT-3, InstructGPT: LLMs que *siguen instrucciones*.
- **2022** — Paper *ReAct*: intercalar razonamiento y acciones.
- **2023** — Explosión de frameworks: LangChain, AutoGPT, BabyAGI.
- **2024-2025** — Tool use nativo, MCP, modelos que “razonan” (o1, DeepSeek-R1), SDKs de agentes de los propios labs.

Evolución de LLM como Herramienta

Modelos Fundacionales

- Predicciones de texto
- Limitaciones: falta de salidas estructuradas

Chats LLM

- Sigue instrucciones conversacionales
- Limitaciones: propenso a alucinaciones, razonamiento limitado

LLM de Razonamiento

- Procesa pasos de manera lógica
- Limitaciones: alto costo computacional

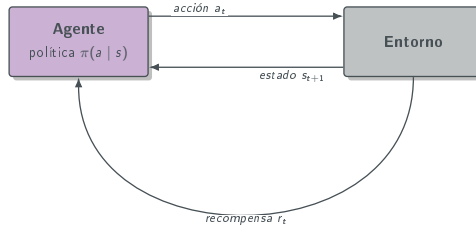
Agentes LLM

- Detecta e integra conocimiento sobre herramientas externas
- Limitaciones: complejidad en implementación

¿Por qué hablar de RL en una charla de agentes LLM?

- La palabra “agente” viene formalmente del **aprendizaje por refuerzo** (Reinforcement Learning, RL).
- RL es el marco matemático donde un agente aprende *qué acción tomar* para maximizar una recompensa acumulada.
- Los LLMs modernos se afinan con una variante de RL (**RLHF**) — por eso el vocabulario de “agente”, “política” y “recompensa” reaparece constantemente.

El ciclo de RL



- El agente busca la política π que **maximiza la suma de recompensas** esperadas a largo plazo.
- Aprende por **prueba y error**: explora, observa el resultado, ajusta su política.

Ingredientes de un problema de RL

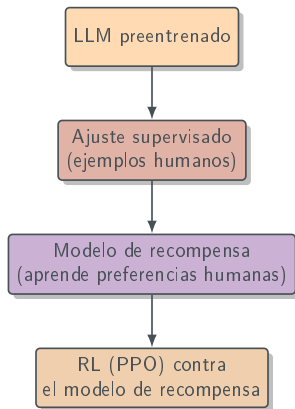
- **Estado (s)**: descripción de la situación actual.
- **Acción (a)**: algo que el agente puede hacer.
- **Recompensa (r)**: señal numérica de qué tan buena fue la acción.
- **Política (π)**: la estrategia que mapea estados a acciones.
- **Valor (V, Q)**: qué tan bueno es un estado (o par estado-acción) a largo plazo.

Ejemplo clásico

Un agente que juega Atari: estado = píxeles de la pantalla, acción = mover el joystick, recompensa = puntaje del juego.

- **Q-learning tabular**: guarda una tabla de valores $Q(s, a)$ — solo funciona con pocos estados posibles.
- **Deep RL** (DQN, PPO, etc.): una red neuronal *aproxima* Q o π directamente — escala a espacios enormes (imágenes, texto).
- Hitos: **DQN** (Atari, 2013), **AlphaGo/AlphaZero** (2016-2017), **OpenAI Five**, **PPO** (algoritmo detrás de RLHF).

El puente hacia los LLM: RLHF



RLHF = *Reinforcement Learning from Human Feedback*. Aquí el “agente” es el propio LLM y la “recompensa” viene de preferencias humanas.

¿En qué se parecen?

- Ambos siguen el ciclo **percibir** → **decidir** → **actuar**.
- Ambos operan dentro de un entorno (juego, sistema operativo, conversación, internet).
- Ambos pueden mejorar con retroalimentación.

¿Entonces qué cambia?

Diagrama comparativo

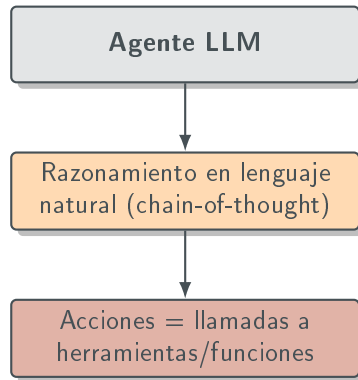
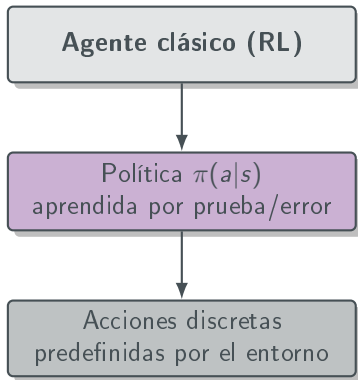


Tabla comparativa

	Agente clásico (RL)	Agente LLM
Cómo decide	Política numérica aprendida	Razonamiento en lenguaje (prompt)
Cómo aprende	Prueba y error con recompensa	Preentrenado en texto + fine-tuning
Espacio de acciones	Fijo, definido por el entorno	Abierto: cualquier herramienta con schema
Adaptación a tareas nuevas	Requiere reentrenar	Cambia con el <i>prompt</i> , sin reentrenar
Explicabilidad	Baja (pesos numéricos)	Alta (razona en texto legible)

Entonces, ¿qué es un agente LLM?

Agente LLM

Un sistema en el que un modelo de lenguaje **decide qué hacer a continuación** —incluyendo qué herramientas invocar y con qué argumentos— de forma iterativa, usando su propio razonamiento en lenguaje natural como “memoria de trabajo”, hasta cumplir un objetivo.

- El LLM cumple el rol de **política**: dado un estado (contexto), produce una acción (texto, tool call).
- Las **herramientas** amplían el entorno con el que puede interactuar (buscar, ejecutar código, leer archivos...).

No todos los LLM son iguales

- El mismo Transformer preentrenado puede terminar en productos muy distintos según cómo se **afine y se use**.
- Distinguimos al menos tres perfiles:
 - ➊ Modelo **base / funcional** (completion).
 - ➋ Modelo **de chat** (instruction-tuned).
 - ➌ Modelo **que razona** (reasoning model).
- Entender la diferencia importa porque **cada uno se comporta distinto dentro de un agente**.

Notebook de ejemplo:

<https://colab.research.google.com/drive/1Q4bxbhWCD-PoDb2inHJTbSmeSjc9g16L?usp=sharing>

1. Modelo base / funcional (completion)

- Entrenado solo para **predecir el siguiente token** sobre un corpus masivo de texto.
- No sabe “seguir instrucciones” ni “conversar” — solo continúa el texto que se le da.
- Ejemplo de entrada: “El capital de Francia es” → continúa con “París.”
- Es la **materia prima**: casi nunca se usa directo en un producto, se afina después.



1. Modelo base / funcional (completion)

- Entrenado solo para **predecir el siguiente token** sobre un corpus masivo de texto.
- No sabe “seguir instrucciones” ni “conversar” — solo continúa el texto que se le da.
- Ejemplo de entrada: “El capital de Francia es” → continúa con “París.”
- Es la **materia prima**: casi nunca se usa directo en un producto, se afina después.

```
classifier("Americans work on <mask>.")  
[{'score': 0.03950061276555061,  
  'token': 12729,  
  'token_str': 'weekends',  
  'sequence': 'Americans work on weekends.'},  
 {'score': 0.03174380958080292,  
  'token': 17429,  
  'token_str': 'Sundays',  
  'sequence': 'Americans work on Sundays.'},  
 {'score': 0.02530151978135109,  
  'token': 2447,
```

```
classifier("Mexicans work on <mask>.")  
[{'score': 0.06137007474899292,  
  'token': 2196,  
  'token_str': 'drugs',  
  'sequence': 'Mexicans work on drugs.'},  
 {'score': 0.032838258892297745,  
  'token': 9485,  
  'token_str': 'farms',  
  'sequence': 'Mexicans work on farms.'},  
 {'score': 0.021477729082107544,  
  'token': 2447,  
  'token_str': 'immigration',  
  'sequence': 'Mexicans work on immigration.'},
```

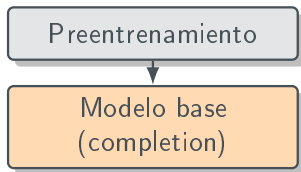
2. Modelo de chat (instruction-tuned)

- Se parte del modelo base y se afina con:
 - Ejemplos de **instrucción** → **respuesta deseada** (SFT).
 - **RLHF/DPO** para preferir respuestas útiles, seguras y bien formateadas.
- Entiende roles (system, user, assistant).
- Es el formato que usan ChatGPT, Claude, Llama-chat, Qwen-instruct...
- Base de casi todos los agentes LLM actuales: necesitas que *siga instrucciones*.

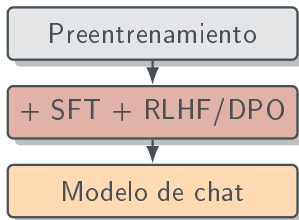
3. Modelo que razona (reasoning model)

- Además del ajuste de chat, se entrena/afina para producir un **razonamiento extendido interno** antes de responder (ej. o1, DeepSeek-R1, Claude con *extended thinking*).
- Ese razonamiento se optimiza (a menudo con RL) para llegar a respuestas correctas en tareas verificables (matemáticas, código).
- Trade-off: **más latencia y costo**, a cambio de mejor desempeño en tareas que requieren varios pasos lógicos.

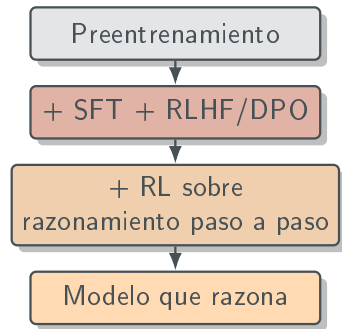
Comparación de los tres pipelines



continúa texto



sigue instrucciones



piensa antes de responder

Un agente LLM típico usa un modelo de **chat** con tool-calling; para tareas complejas de varios pasos conviene un modelo **que razona**.

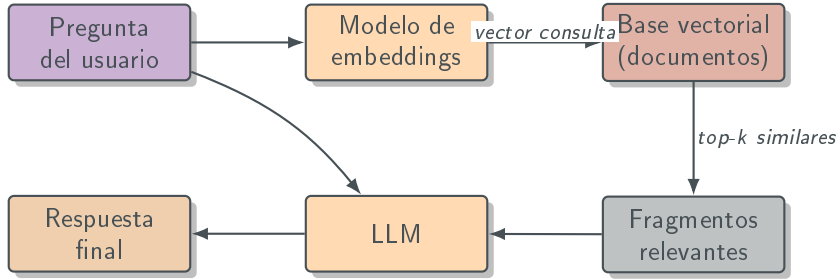
El problema que resuelve RAG

- Un LLM solo “sabe” lo que vio en su entrenamiento: conocimiento **congelado en el tiempo** y sin datos privados/internos.
- Alternativas para darle información nueva:
 - 1 Reentrenar/afinar el modelo (caro, lento).
 - 2 Meter todo en el prompt (no escala, límite de contexto).
 - 3 **RAG**: buscar solo lo relevante y ponerlo en el prompt.

RAG

Recuperar información relevante de una fuente externa *en el momento de la consulta* y dársela al LLM como contexto para generar la respuesta.

Pipeline de RAG



La pregunta viaja por dos caminos: se usa para **buscar** y se usa (junto al contexto recuperado) para **generar** la respuesta.

¿Qué es un embedding, en una frase?

Embedding

Un vector de números que representa el *significado* de un texto, de forma que textos con significado parecido quedan cerca en ese espacio vectorial.

- Se indexan todos los documentos (o fragmentos/*chunks*) como embeddings en una **base vectorial** (FAISS, Chroma, pgvector...).
- En consulta: se calcula el embedding de la pregunta y se busca por **similitud** (coseno) los fragmentos más cercanos.

Ejemplo 1: asistente sobre documentación interna

- 1 Se indexan los manuales/wiki de la empresa en chunks de ~500 tokens.
- 2 Usuario pregunta: “¿*Cuántos días de vacaciones tengo el primer año?*”
- 3 Se recuperan los 3 fragmentos del manual de RR.HH. más relevantes.
- 4 El LLM responde **citando** esos fragmentos, sin inventar la política.

Por qué importa

Sin RAG, el modelo alucinaría una cifra genérica; con RAG, responde con la política *real* de la empresa.

Ejemplo 2: RAG dentro de un agente

- 1 El agente recibe: *“Resume los cambios de la última versión de nuestra librería”*.
- 2 El agente decide llamar a la herramienta `buscar_docs(query)`.
- 3 La herramienta hace la recuperación (RAG) y devuelve fragmentos del changelog.
- 4 El agente redacta el resumen usando esos fragmentos como evidencia.

RAG aquí **no es un paso fijo**: es una herramienta más que el agente decide usar cuando la necesita.

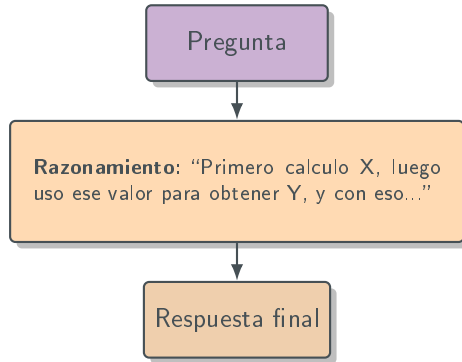
- **Chunking**: fragmentos muy grandes pierden precisión, muy chicos pierden contexto.
- **Recuperación pobre** = respuesta pobre, aunque el LLM sea excelente.
- Combinar búsqueda **semántica** + **por palabras clave** (*hybrid search*) suele dar mejores resultados.
- *Re-ranking*: recuperar más candidatos y luego reordenar los mejores con un modelo más preciso (y más caro).

Chain-of-Thought

Técnica de *prompting* donde se pide (o se induce) al modelo a escribir su **razonamiento paso a paso** antes de dar la respuesta final, en lugar de responder directo.

- Simple de aplicar: basta agregar “pensemos paso a paso” al prompt.
- Mejora notablemente el desempeño en tareas de **lógica, matemáticas y problemas de varios pasos**.
- Es la base conceptual de los modelos “que razonan” (sección anterior).

CoT en un diagrama



CoT es puro *razonamiento en texto* — todavía no interactúa con el mundo exterior.

- CoT razona muy bien **sobre lo que ya sabe**.
- Pero no puede verificar hechos, hacer cálculos exactos grandes, ni consultar información nueva — solo “piensa en voz alta”.
- ¿Qué pasa si el razonamiento necesita un dato externo (una búsqueda, un cálculo, un archivo)?

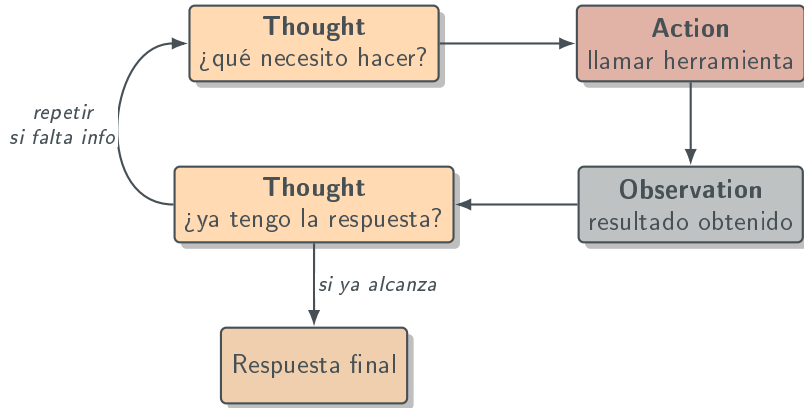
⇒ ReAct

ReAct (Yao et al., 2022)

Intercalar pasos de **razonamiento** (Thought) con pasos de **acción** (Action) sobre herramientas externas, observando el resultado (Observation) antes de continuar razonando.

Thought → Action → Observation → Thought → ... → Respuesta final

El loop ReAct



Pregunta

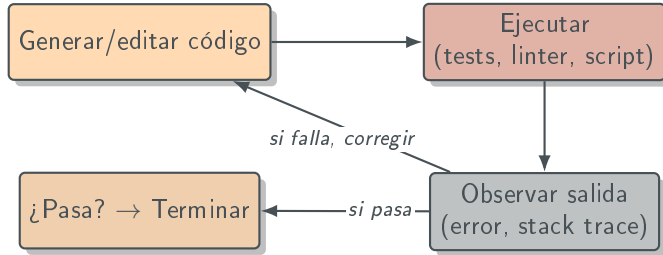
“¿Qué día de la semana cae el 25 de diciembre de este año, y a cuánto equivale en horas si faltan 40 días?”

- Thought: necesito saber la fecha de hoy.
- Action: `fecha_actual()`
- Observation: "2026-07-02"
- Thought: ahora calculo los días restantes y la conversión.
- Action: `calculadora("40 * 24")`
- Observation: 960
- Thought: ya puedo responder.
- **Respuesta final:** “Faltan 960 horas; el 25 de diciembre es viernes.”

De autocompletado a agente de código

- **Autocompletado** (Copilot clásico): sugiere la siguiente línea, el humano decide todo lo demás.
- **Agente de código** (Claude Code, Cursor Agent, Devin...): puede *planear, escribir, ejecutar, ver el error y corregir* — todo en un loop, con supervisión humana en distintos grados.
- La diferencia clave: el agente tiene **herramientas** para leer/escribir archivos y **ejecutar** lo que escribe.

El loop generar-ejecutar-corregir



Este es, literalmente, ReAct aplicado a ingeniería de software: **Thought** (qué cambiar) → **Action** (editar/ejecutar) → **Observation** (resultado).

Ejemplos de tareas automatizables

- **Refactors mecánicos a escala:** renombrar una función y actualizar todos sus usos en un repo grande.
- **Corrección de bugs guiada por tests:** correr la suite, leer el fallo, editar, repetir hasta que pase.
- **Generación de código repetitivo:** boilerplate, migraciones, adaptadores entre APIs.
- **Revisión de código:** detectar bugs, sugerir simplificaciones antes de mergear.
- **Automatización de CI/CD:** diagnosticar por qué falló un pipeline y proponer el fix.

Qué necesita un agente para hacer esto bien

- Acceso de **lectura/escritura** al sistema de archivos.
- Una forma de **ejecutar comandos** (shell, tests, build).
- **Feedback verificable**: tests, linters, tipos — señales objetivas de si el cambio funcionó (no solo “se ve bien”).
- **Límites y supervisión**: permisos, sandboxing, confirmación humana en acciones riesgosas (borrar, hacer push, etc.).

No es magia, hay que vigilarlo

- Puede “arreglar” el síntoma y no la causa si el feedback es débil.
- Puede ejecutar comandos destructivos si no hay barreras.
- Requiere buen criterio sobre **qué automatizar sin supervisión** y qué requiere aprobación humana explícita.

¿Para qué sirve un framework de agentes?

- Resuelven en común: manejo de historial, parsing de tool calls, loops de agente, memoria, orquestación multiagente.
- No son estrictamente necesarios (un loop ReAct se puede escribir a mano, como veremos en la demo) — pero ahorran trabajo repetitivo en proyectos grandes.
- Riesgo: añaden **abstracción y dependencias**; a veces conviene empezar simple y adoptar un framework solo cuando se necesita.

Panorama de frameworks (2026)

Framework		Enfoque principal
LangChain	/	Cadenas de LLM + grafos de estados explícitos para agentes complejos
LangGraph		
CrewAI		Equipos de agentes con roles definidos (multiagente)
AutoGen	/ AG2	Conversaciones entre múltiples agentes
(Microsoft)		
OpenAI	Agents	Agentes ligeros con <i>handoffs</i> entre ellos
SDK		
Claude	Agent	El mismo motor de agente detrás de Claude Code, reusable en tus apps
SDK		

¿Framework o loop a mano?

Tarea simple, 1 agente, pocas herramientas



→ loop ReAct escrito a mano

Multiagente, memoria persistente, orquestación



→ framework (LangGraph, CrewAI...)

En esta charla construimos el caso simple: entender el loop antes de delegarlo a un framework.

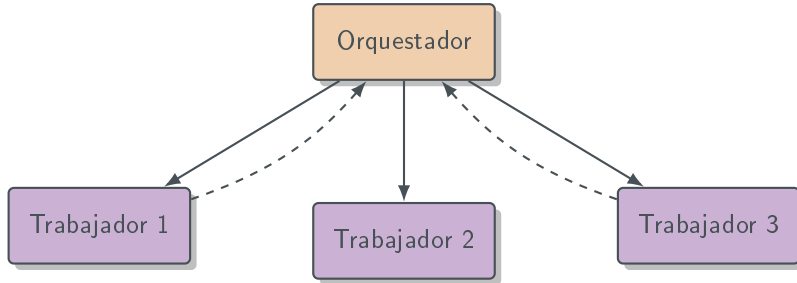
Protocolo común: Model Context Protocol (MCP)

- Estándar abierto (Anthropic, 2024) para conectar un LLM con **herramientas y fuentes de datos** de forma uniforme.
- Antes: cada integración era un conector distinto por framework.
- Con MCP: un servidor de herramientas puede usarse desde cualquier cliente/agente compatible (Claude, editores, otros frameworks).
- Ayuda a que el ecosistema de herramientas **no dependa de un solo framework**.

¿Por qué usar varios agentes?

- Un solo agente con demasiadas responsabilidades tiende a **perder foco** (contexto saturado, instrucciones contradictorias).
- Dividir el trabajo entre agentes **especializados** —cada uno con su propio prompt, herramientas y contexto— suele dar mejores resultados en tareas grandes.
- Costo: más complejidad, más latencia, más tokens.

Topología 1: orquestador-trabajadores



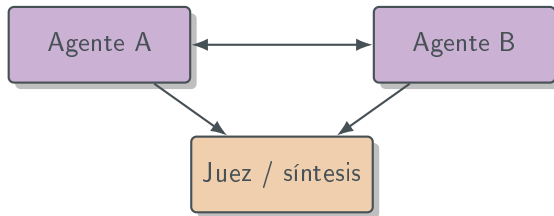
Un agente central descompone la tarea, la reparte y **combina los resultados**. Ideal cuando las subtareas son independientes entre sí.

Topología 2: pipeline secuencial



Cada agente recibe la salida del anterior como entrada. Bueno para procesos con **etapas claras** (investigar → escribir → corregir).

Topología 3: colaborativa / debate



Los agentes discuten/critican propuestas entre sí antes de llegar a una respuesta final — útil para **decisiones que se benefician de puntos de vista distintos**.

Riesgos propios de multiagente

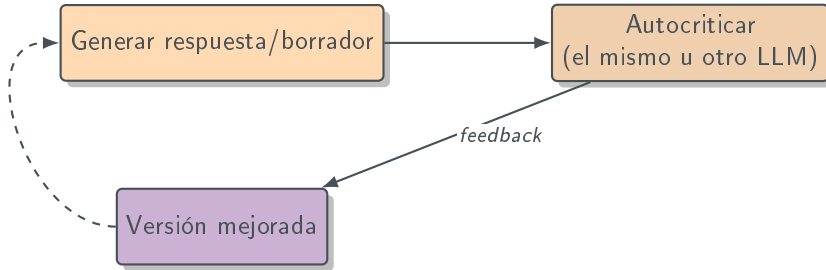
- **Costos y latencia** se multiplican por cada agente involucrado.
- **Errores en cascada**: un agente con información incorrecta contamina a los que dependen de él.
- **Coordinación**: decidir *cuándo termina* cada agente y qué formato usan para comunicarse entre sí.
- **Regla práctica**: usar multiagente solo cuando un solo agente **ya demostró** no ser suficiente.

Cuatro patrones recurrentes

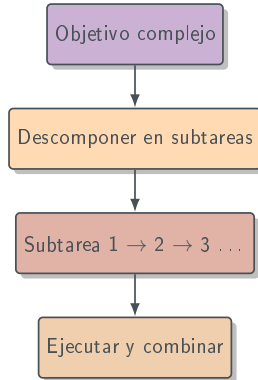
- ➊ **Reflection** (auto-crítica)
- ➋ **Planning** (planeación)
- ➌ **Tool use** (uso de herramientas)
- ➍ **Multi-agent collaboration** (colaboración multiagente)

Popularizados por Andrew Ng / DeepLearning.AI como los bloques básicos con los que se construye casi cualquier sistema agéntico.

Patrón: Reflection

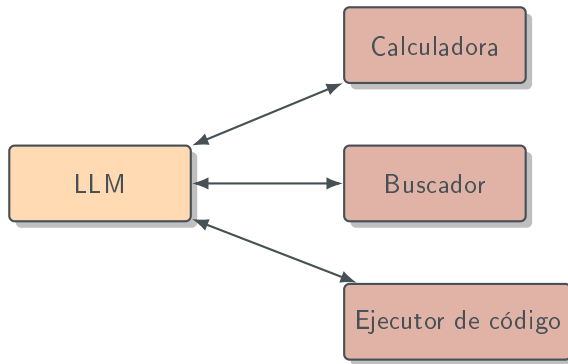


El agente revisa su propio trabajo antes de entregarlo — como un borrador y su edición.



El agente traza un plan *antes* de actuar, en vez de improvisar acción por acción.

Patrón: Tool use



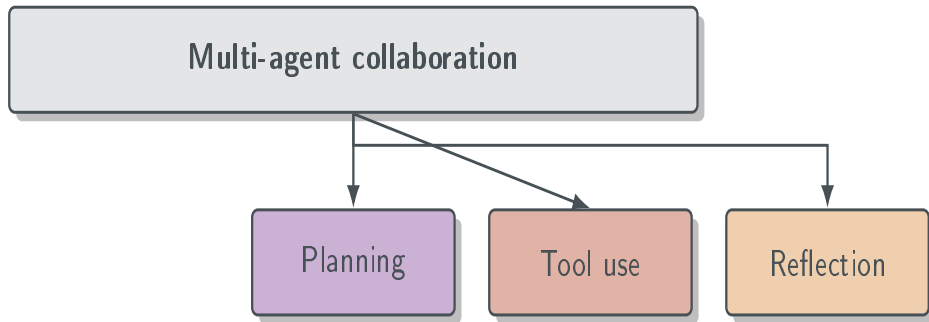
El patrón que ya vimos en RAG y ReAct: el LLM decide *cuándo* y *con qué argumentos* invocar cada herramienta.

Patrón: Multi-agent collaboration

Ya visto en la sección anterior: orquestador-trabajadores, pipeline, debate.

- Se combina con los otros tres: cada agente puede tener su propio ciclo de *planning + tool use + reflection*.
- Es el patrón de **mayor nivel**: compone a los demás.

Los cuatro patrones combinados



En la práctica, un buen agente combina varios de estos patrones a la vez, no uno solo.

¿Qué es una “herramienta” para un LLM?

Tool / function calling

Una función externa que el LLM puede invocar, descrita mediante un **schema** (nombre, descripción, parámetros) que el modelo usa para decidir cuándo y cómo llamarla.

- El LLM **no ejecuta la función**: solo genera la *intención* de llamarla (nombre + argumentos en JSON).
- El programa que envuelve al LLM es quien **ejecuta** la función real y le devuelve el resultado.

Anatomía de un tool schema

```
{  
  "name": "calculadora",  
  "description": "Evalua una expresion aritmetica",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "expresion": {"type": "string"}  
    },  
    "required": ["expresion"]  
  }  
}
```

El modelo elige el nombre y llena los parámetros; tu código ejecuta `calculadora(expresion="3*7")`.

Categorías de herramientas comunes

- **Cómputo:** calculadora, intérprete de código (Python, SQL).
- **Información:** búsqueda web, RAG sobre documentos, APIs.
- **Sistema de archivos:** leer/escribir/editar archivos.
- **Acciones externas:** enviar correos, crear tickets, llamar APIs de terceros.
- **Memoria:** guardar/recuperar notas entre sesiones.

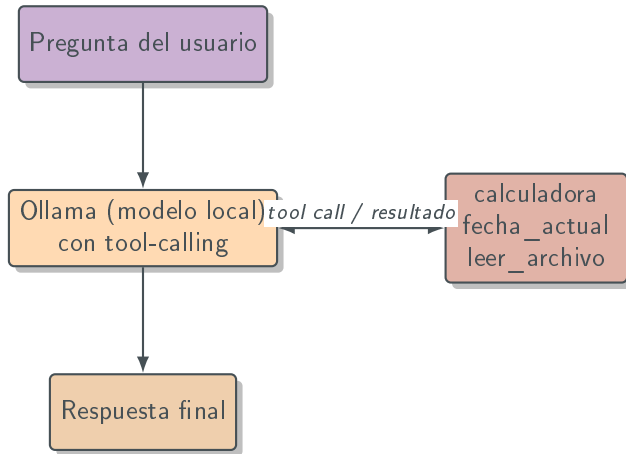
MCP: estandarizando las herramientas

- **Model Context Protocol:** un servidor MCP expone un set de herramientas de forma estándar, consumible por cualquier cliente compatible.
- Ventaja: escribes la herramienta **una vez**, se conecta a Claude, a un IDE, o a cualquier agente que hable MCP.
- Es el equivalente a lo que USB hizo con los periféricos: un conector común en vez de uno distinto por dispositivo.

Lo que vamos a construir

- Un agente **mínimo** corriendo 100 % local con Ollama — sin llaves de API, sin costo.
- Tres herramientas simples: `calculadora`, `fecha_actual`, `leer_archivo`.
- El ciclo **ReAct** que ya vimos, implementado en ~100 líneas de Python: sin frameworks, para entender cada pieza.

Arquitectura de la demo



Dos formas de seguir la demo

Script

`codigo/agente_simple.py`

Ejecución de principio a fin, para correr en terminal.

Notebook

`notebook/agente_llm_local.ipynb`

Celda por celda, con explicación antes de cada paso.

Vamos al código.

¡Gracias!

Preguntas y discusión

ulises.moya@iieg.gob.mx